

SRS — Especificación de Requisitos de Software: Kinema

Documento de ingeniería inversa (IEEE 830 simplificado), generado por análisis del sistema implementado. Cubre el estado real verificado en código, rutas, esquema y datos (sep-2026).

1. Introducción

1.1 Propósito

Definir qué hace Kinema, para quién y bajo qué restricciones, como base de mantenimiento, onboarding y auditoría.

1.2 Alcance

Red social de cine con catálogo enriquecido por ML: usuarios registran películas vistas, califican con estrellas, reseñan, comentan, arman listas públicas/privadas, siguen a otros y reciben recomendaciones por similitud semántica (pgvector) y vibras (clusters K-Means etiquetados con Gemini). Incluye importación del historial de Letterboxd (ZIP) y analítica (sentimiento NLP en español, heatmaps, radares, taquilla TMDb).

Fuera de alcance: apps móviles nativas, pagos, moderación automatizada, series/TV (TMDb solo endpoint de películas).

1.3 Definiciones

- **Vibra**: cluster semántico de películas (K-Means sobre embeddings Gemini).
- **Slug (lista)**: identificador URL único por usuario (/usuario/lists/slug).
- **Pulso**: conteo POSITIVE/NEGATIVE por película (pipeline BETO español).
- **Stub**: película creada por importación sin tmdb_id, converge al catálogo.

1.4 Referencias

README.md, docs/architecture.md, docs/er.md, docs/diagrams.md, routes/api.php, backend/python/requirements.txt.

2. Descripción general

2.1 Actores

Actor	Descripción
Visitante	Navega catálogo, búsqueda, fichas, listas públicas, perfiles. Sin escritura.
Miembro	Registrado con token Bearer. Reseña, comenta, listas, follows, importa ZIP.

Actor	Descripción
Pipeline	Script Python con token de servicio. Lee reseñas, escribe JSON de sentimiento.
TMDB	API externa (fichas, posters, cast, taquilla). Solo lectura.

2.2 Supuestos y dependencias

- PostgreSQL 16 + extensión pgvector (recomendaciones y similares la exigen).
- Claves externas: TMDB_API_KEY, GENAI_API_KEY, KINEMA_API_TOKEN.
- Frontend SSR contra PUBLIC_API_URL; CORS restringido a orígenes declarados.

3. Requisitos funcionales

RF-01 Autenticación y sesión

- RF-01.1 Registro con nombre/email/contraseña (mín. 8, confirmada); crea perfil y devuelve Bearer token.
- RF-01.2 Login por email/contraseña; devuelve token + usuario con perfil.
- RF-01.3 Logout revoca el token actual. Sesión persistida en localStorage.
- RF-01.4 Rutas de escritura exigen auth: sanctum; las de otros exigen propiedad.

RF-02 Catálogo y descubrimiento

- RF-02.1 Catálogo paginado (20) con vibra; detalle con sinopsis, reparto, stats, sentimiento, similares y taquilla.
- RF-02.2 Búsqueda unificada q + filtros vibe, year_from/to, genre, person; incluye listas **públicas** y personas.
- RF-02.3 Similares por distancia coseno pgvector ($\leq$), top 5, excluye sin embedding.
- RF-02.4 Recomendaciones: anónimas = tendencia; autenticadas = similares a sus 4-5★ + vibras top + relleno.

RF-03 Diario social (reseñas)

- RF-03.1 Una reseña por usuario/película (updateOrCreate); rating 0.5-5.0, texto $\leq$ 2000, spoiler flag, watched_at opcional.
- RF-03.2 Editar/borrar propios. Estrellas 1-5 guardan sin exigir texto; el texto no borra la nota (merge por campo presente).
- RF-03.3 Comentarios sobre reseñas (CRUD propio), paginados, con autor.
- RF-03.4 Diario (/diary) y GET /users/{id}/reviews con película y póster.

RF-04 Listas

- RF-04.1 CRUD de listas con slug único por usuario (auto desde nombre + sufijo).
- RF-04.2 Añadir/quitar películas (orden sort_order, idempotente).
- RF-04.3 Visibilidad pública/privada; las privadas solo las ve el dueño (auth opcional en lectura).
- RF-04.4 URL canónica /{usuario}/lists/{slug} con items + película anidada.

RF-05 Red social

- RF-05.1 Follow/unfollow con toggle (auto-follow prohibido), contadores y estado.
- RF-05.2 Feed con últimas 20 reseñas + listas públicas de seguidos, orden global.
- RF-05.3 Perfiles por id y por username, con contadores y Top 4 editable.

RF-06 Importación Letterboxd

- RF-06.1 POST /import/letterboxd acepta el ZIP tal cual ($\leq 20\text{MB}$): ratings, diary, watched, reviews (texto multilínea), watchlist, lists/*.csv (usa el nombre de archivo como slug), likes/films, profile.csv. Ignora deleted/, orphaned/ y likes ajenos. Transacción atómica.
- RF-06.2 Matcheo título+año → fallback a fila sin fechas → stub; enriquecimiento TMDB inmediato con fusión a canónica (mueve reseñas/items, borra duplicado).

RF-07 Enriquecimiento TMDB

- RF-07.1 Posters, sinopsis, géneros, fechas, cast (top-15) + dirección/escritura, bio de artistas bajo demanda, budget/revenue/runtime.
- RF-07.2 Match con scoring (título exacto/original + año + popularidad, umbral mínimo); sin match fuerte no se escribe basura.
- RF-07.3 Comandos app:enrich-posters (--only-missing, --force, --ids) y app:enrich-credits.

RF-08 Analítica

- RF-08.1 movieSentiment por película desde JSON del pipeline (tolerante a ausencias y claves extra).
- RF-08.2 Actividad por día, radar de vibras, mapa x/y (500), desglose de géneros (normaliza nombres e ids TMDB), histograma de ratings en el detalle.
- RF-08.3 analytics:refresh ejecuta el pipeline Python (venv con torch CPU + pysentimiento/BETO español).

4. Requisitos no funcionales

ID	Requisito	Estado medido
RNF-01	API p95 inferior a 1 s en red local	/up 0.014s, /movies 0.1s, search 0.05s [OK]
RNF-02	Secretos fuera de git	.env ignorados; opencode.json solo {env:...} [OK]
RNF-03	Sin N+1 críticos	with/withCount en listados [OK]
RNF-04	Degradación elegante	Fallbacks a mock si el API cae; 404/403 JSON [OK]

ID	Requisito	Estado medido
RNF-05	Privacidad de listas	Privadas invisibles en búsqueda y lectura anónima [OK]
RNF-06	Portabilidad dev	Sail + volúmenes nombrados (sail-pgdata fija el PGDATA real) [OK]

5. Restricciones y deuda conocida

- `main.py` cubre catálogo; el refresh de sentimiento es por script directo (el contenedor Sail no tiene el venv).
- 3 películas sin sinopsis y 1 sin póster (sin match TMDB, curaduría manual).
- `World on a Wire` solo existe como serie en TMDB (sin soporte TV).
- Sin rate-limit ni caché HTTP en el API (aceptable en dev con ~500 filas).